Comparação: Next.js vs. Opção B (Vite) para o Menu Play

> Análise comparativa entre migrar o projeto para **Next.js** ou seguir a **Opção B (Vite + React 18 + MUI v5 + RR6)** recomendada em `ATUALIZACAO.md`.

---

1. A Decisão Pivota em 2 Perguntas

1. **SEO importa para este produto?** (cardápios de restaurantes serem indexados no Google)
2. **Você quer preservar o código existente ou está disposto a refatorar o shell da app inteiro?**

O Next.js vence em (1) e perde em (2) para este projeto específico.

---

2. Impacto no Código Existente (o ponto decisivo)

Componente do projeto atual	Opção B (Vite)	Next.js (App Router)
`routes.js` (Switch/Route)	Preserva, só bumps v5→v6	**Reescreve** (file-based routing, layouts, loading states)
`PrivateRoute` (`auth.js`)	Preserva lógica	**Reescreve** (Server Components não suportam `Parse.User.current()` assíncrono — necessita `'use client'` boundary)
`useEffect` de carregamento do Parse	Preserva	Cada um precisa de `'use client'` ou refatorar para Server Component com `fetch` server-side
`useCart` (localStorage)	Preserva	Precisa de `'use client'` (OK, mas boundary)
`makeStyles` do MUI v4 → v5	Mesma mecânica em SPA	**Mais complexo** — MUI v5 + Next App Router tem problemas conhecidos com emotion SSR (precisa `AppRouterCacheProvider`)
~1500 linhas de páginas CRUD e carrinho	Preservam	Maioria preserva, mas com `'use client'` espalhado e ajustes
Form de checkout com `useState` multi-step	Preserva	Precisa `'use client'`

Estimativa de retrabalho

- Opção B: ~10% do código reescrito (shell + rotas)
- Next.js: ~30-40% reescrito (shell + auth + layout + ajustes de cliente/servidor em cada página)

---

3. Comparação por Dimensão

Dimensão	Opção B (Vite+R18+MUI5+RR6)	Next.js 14 (App Router)
SEO da landing	Client-rendered, SEO fraco	SSR/SSG, SEO real
SEO do cardápio /:subdominio/menu	Fraco	SSR dinâmico por empresa
Otimização de imagens dos produtos	Manual (sem next/image)	Nativo `next/image`
Multi-tenant subdomínio real (não path)	Reescrever DNS + lógica	Middleware do Next resolve
Comportamento com Parse SDK	Compatível (client-side puro)	Client components + boundary
Esforço de migração	7-11 dias	12-18 dias
Dev server	Vite HMR instantâneo	Fast Refresh
Bundle	SPA único + code-split por rota	Por rota + streaming
Curva para novo dev	Padrão SPA conhecido	Server/Client Components (conceito extra)
Manutenção a longo prazo	Precisa cuidar de rotas/auth	Framework cuida de mais coisas
Deploy	SPA estático (qualquer host)	Node.js runtime ou Vercel

4. Onde o Next.js Realmente Justificaria a Troca

1. **SEO dos cardápios é requisito de negócio** — restaurantes precisam ser encontrados no Google. Hoje `/pizzaria-do-joao/menu` renderiza no client somente — Google não vê os produtos no HTML. SSR resolve isso e é o argumento comercial real a favor do Next.
2. **Trocar do Parse para API própria algum dia** — se a ideia for sair do back4app, o Next API Routes vira o backend. Hoje não há plano de sair do Parse.
3. **Subdomínio real** (`pizzaria.menuplay.com` em vez de `/pizzaria/menu`) — o middleware do Next lida com isso. Hoje o projeto usa path, não subdomínio.

Nenhum desses três estava no escopo original do projeto. A função de `subdominio` no Parse sequer é configurada como DNS real — é só uma string no path.

> **Nota sobre subdomínio real:** é possível implementar em React puro (Vite), não é exclusivo do Next. Exige DNS wildcard (`*.menuplay.com`), servidor web wildcard (Nginx `server_name *.menuplay.com`) e SSL wildcard (Let's Encrypt DNS-01) — uma corrente de 3 camadas de infraestrutura. O Next não elimina essa infra, só empacota a leitura do subdomínio no middleware. Em SPA, troca-se `useParams().subdominio` por um hook que lê `window.location.hostname`. O trabalho real está na infra, não no framework.

5. Onde a Opção B Ganha de Longe

1. **Preserva 90% do código que funciona** — toda a lógica de páginas Parse, callbacks de useEffect, PrivateRoute com `Parse.User.current()` continua válida.
2. **Migração mecânica** — Vite + React 18 + RR6 são bumps de versão com mudanças de API localizadas.
3. **Esforço ~50% menor** para chegar no mesmo ponto funcional.
4. **Curso natural de um dev novo entender** — SPA puro é padrão ainda dominante em educação/trabalho.

---

6. Veredito

Para a situação atual deste projeto: Opção B (Vite)

Justificativa:

- O ganho de SEO do Next só vale se o requisito de negócio explicitar "cardápios indexáveis no Google". Não foi levantado.
- O custo de migrar para Next App Router é ~50% maior e força reescrever `PrivateRoute`, `routes.js` e adicionar `'use client'` boundaries em todas as páginas com estado.
- O Parse SDK é client-first por design — encaixe natural em SPA, atrito real com SSR.
- 70% do projeto já está em padrão SPA. Reescrever o shell para pegar SSR que não se usa é desperdício.

Quando reconsiderar Next no futuro

- Se SEO de cardápio virar requisito de negócio documentado, ou se decidir sair do Parse para ter backend próprio. Aí o investimento compensa. **Não antes.**

Recomendação completa

Execute o `ATUALIZACAO.md` rota B. Se mais adiante SEO for requisito, avaliar Next como **fase futura** — não agora.